

아직도 직접 구현중?

.NET

LLM 직접 구현할 시간에
난 짬x치킨 간다

박경선 | enfj.dev@gmail.com

LinkedIn : @gyeongsun-park

Github : @gngsn



Contents

- 01 OpenChat Playground?
- 02 OpenChat Playground 실행 🚀
- 03 Microsoft.Extensions.AI (*a.k.a. MEAI*)
- 04 MEAI – Key Idea 💡
- 05 Flow: Web UI → External LLM Provider
- 06 Roadmap 🚗 & Open Source Contribution



AI

LLM

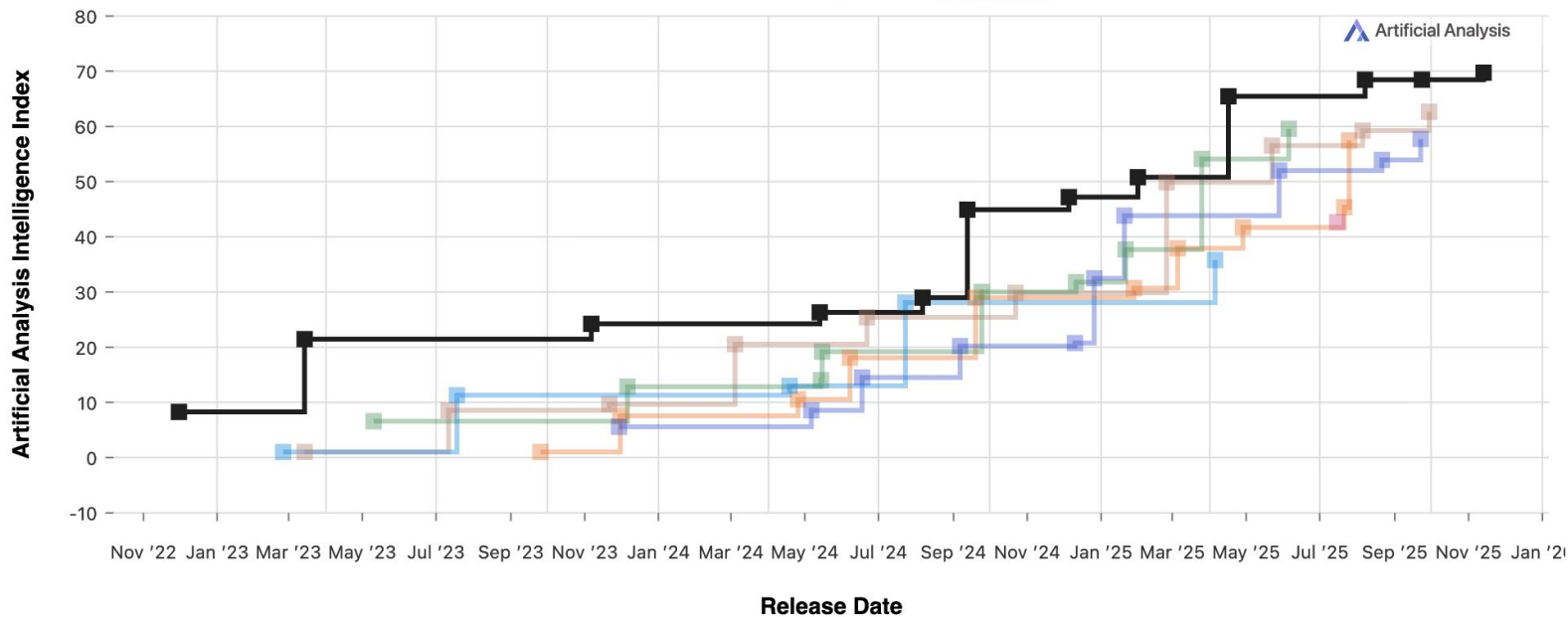




Frontier Language Model Intelligence, Over Time

Artificial Analysis Intelligence Index v3.0 incorporates 10 evaluations: MMLU-Pro, GPQA Diamond, Humanity's Last Exam, LiveCodeBench, SciCode, AIME 2025, IFBench, AA-LCR, Terminal-Bench Hard, τ^2 -Bench Telecom

Alibaba Anthropic DeepSeek Google LG AI Research Meta OpenAI





Introducing the
world's most
powerful model



Introducing the
world's most
powerful model



Introducing the
world's most
powerful model



Introducing the
world's most
powerful model



 학습 시간

 구현 시간

 테스트

 배포

독자적인 구현 및 실행 방식을 가진 모델

ChatGPT



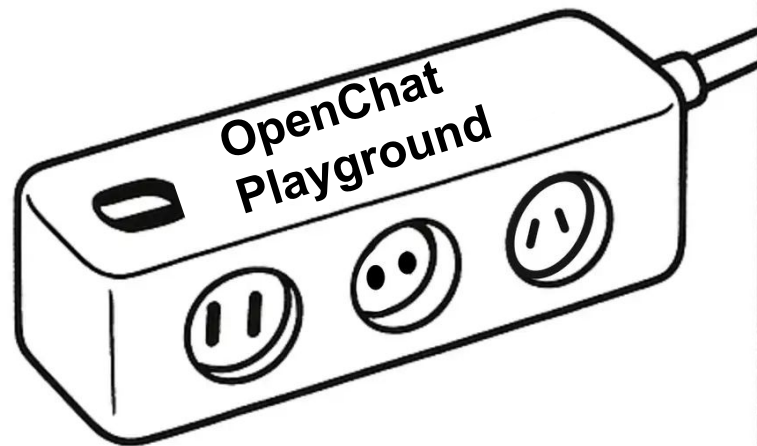
Claude



Gemini



OpenChat
Playground





OpenChat Playground

<https://bit.ly/openchat-playground>



aliencube/open-chat-playground

github.com/aliencube/open-chat-playground

Relaunch to update

READMELicense

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.

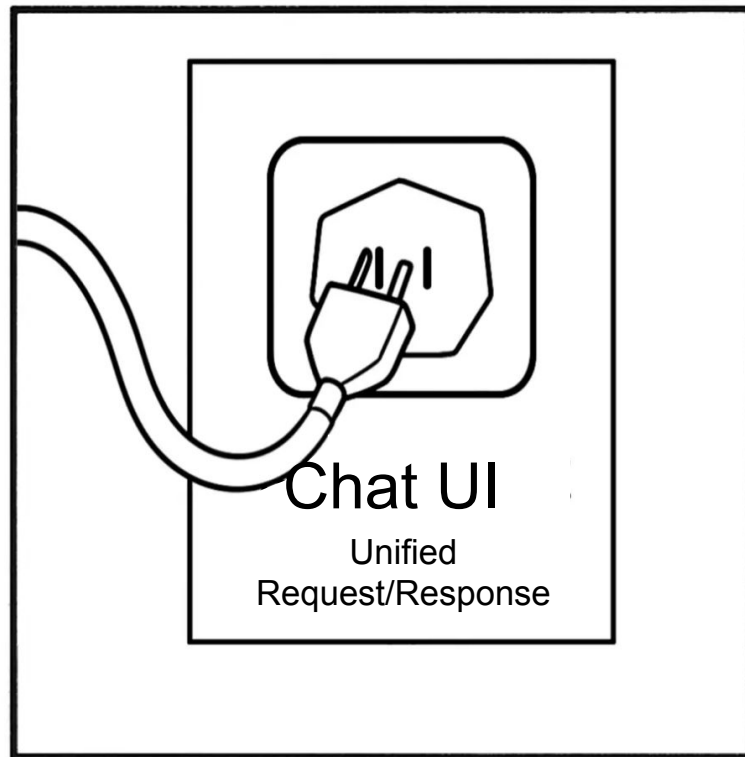
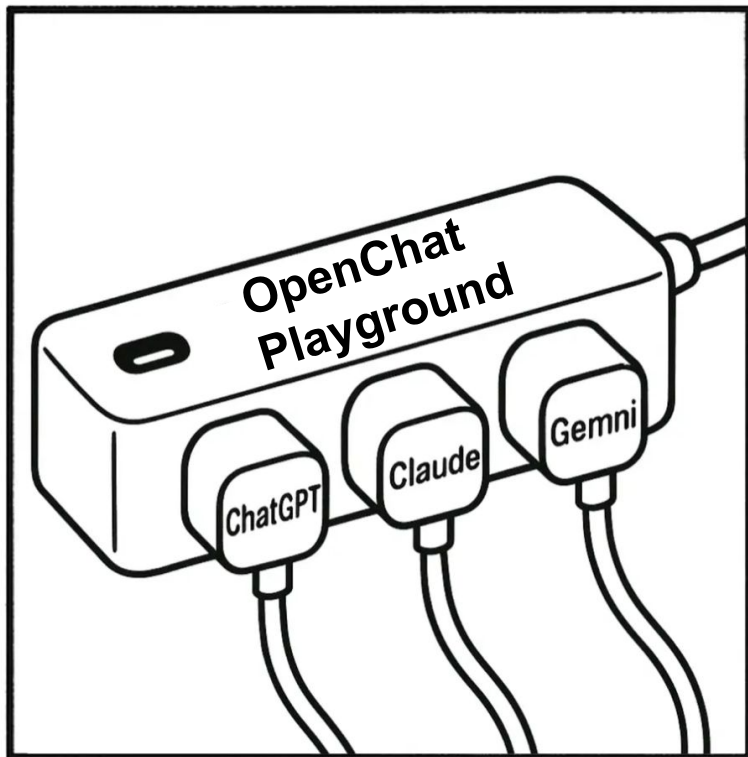
The diagram illustrates the Open Chat Playground (OCP) architecture. On the left, the Blazor and Microsoft.Extensions.AI logos are shown. A central box contains three categories of supported models: 'Model As A Service' (Amazon Bedrock, Azure AI Foundry, GitHub Models, Google VertexAI), 'Local' (Docker Model Runner, Foundry Local, HuggingFace, Ollama), and 'Vendor' (Anthropic Claude, LG AI Exaone, Never HyperCLOVA X, OpenAI GPT, Upstage Solar). The OCP logo is at the bottom of this central box.

Languages

C# 86.2%	JavaScript 8.2%
Bicep 2.6%	CSS 1.4%
HTML 1.3%	Shell 0.2%
Dockerfile 0.1%	

Supported platforms

- ☒ [Amazon Bedrock](#)
- ☒ [Azure AI Foundry](#)



LLM Providers

© Open Chat Playground

Model As A Service



Amazon
Bedrock



Azure
AI Foundry



Github
Models



Google
VertexAI

LLM Providers

© Open Chat Playground

Model As A Service



Amazon
Bedrock



Azure
AI Foundry



Github
Models



Google
VertexAI

Local



Docker
Model Runner



Foundry
Local



HuggingFace



Ollama

LLM Providers

© Open Chat Playground

Model As A Service



Amazon
Bedrock



Azure
AI Foundry



Github
Models



Google
VertexAI

Local



Docker
Model Runner



Foundry
Local



HuggingFace



Ollama

Vendor



Anthropic
Claude



LG
AI Exaone

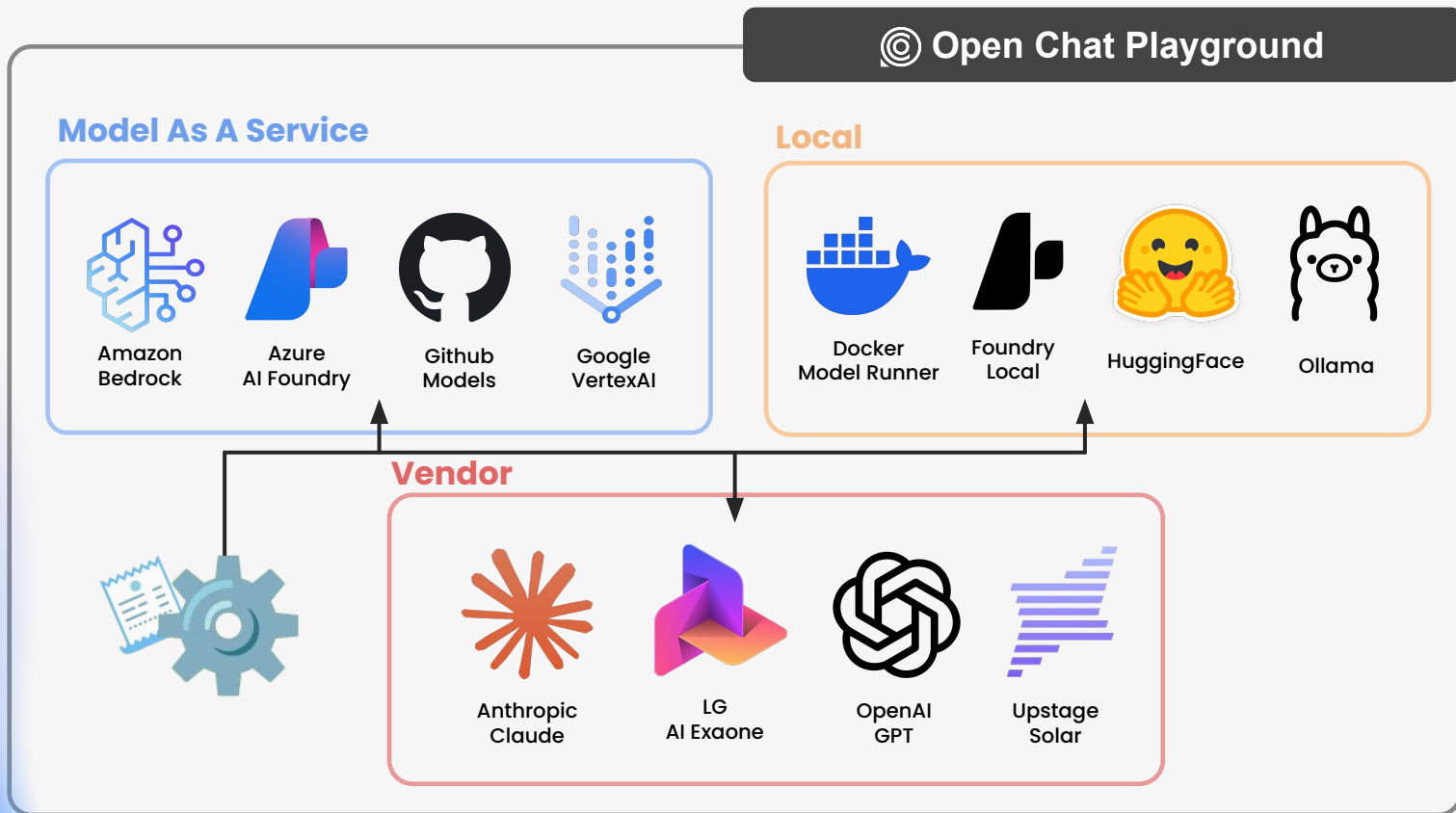


OpenAI
GPT



Upstage
Solar

LLM Providers



쉬운 LLM 교체 —



빠른 최신 모델 적용



벤더 종속성 방지 Vendor Lock-in 회피



자유로운 실험 · A/B 테스트



비용·용도에 최적화된 모델 선택

쉬운 LLM 교체 —



빠른 최신 모델 적용



벤더 종속성 방지 Vendor Lock-in 회피



자유로운 실험 · A/B 테스트



비용·용도에 최적화된 모델 선택

쉬운 LLM 교체 —



빠른 최신 모델 적용



벤더 종속성 방지 Vendor Lock-in 회피



자유로운 실험 · A/B 테스트



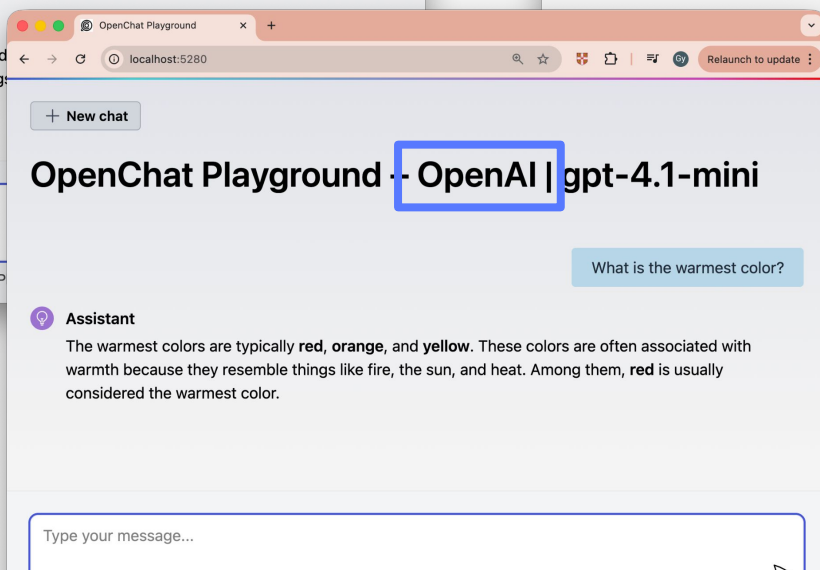
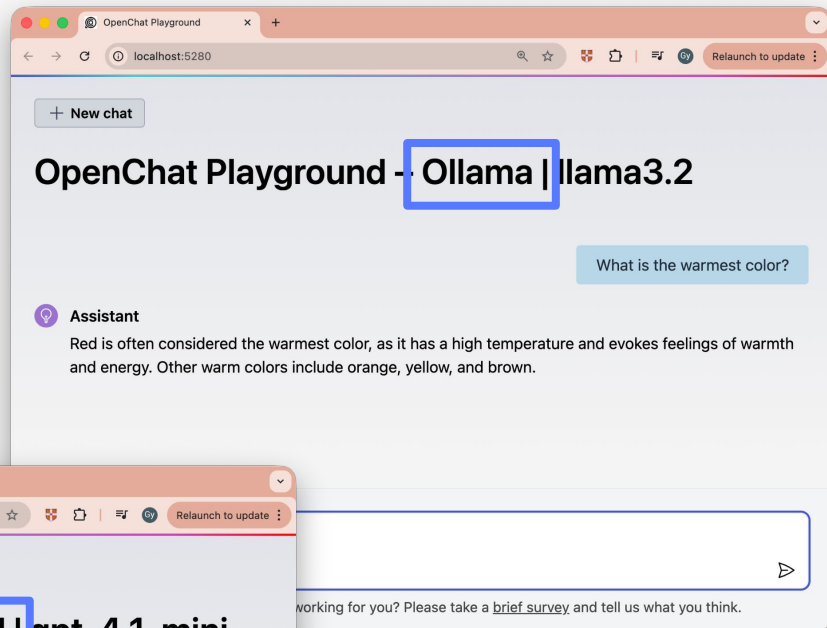
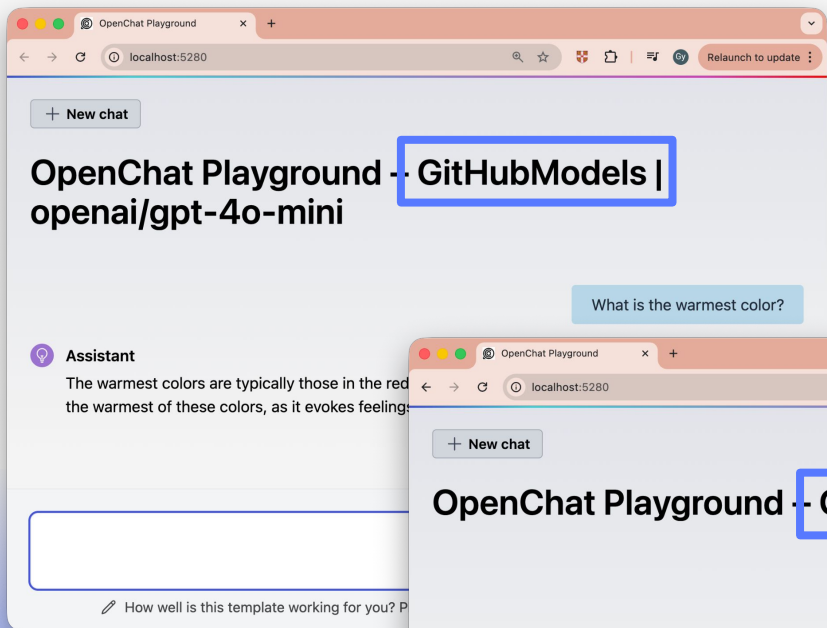
비용·용도에 최적화된 모델 선택

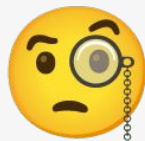
쉬운 LLM 교체 —

- ✓ 빠른 최신 모델 적용
- ✓ 벤더 종속성 방지 Vendor Lock-in 회피
- ✓ 자유로운 실험 · **A/B 테스트**
- ✓ 비용·용도에 최적화된 모델 선택

쉬운 LLM 교체 —

- ✓ 빠른 최신 모델 적용
- ✓ 벤더 종속성 방지 Vendor Lock-in 회피
- ✓ 자유로운 실험 · A/B 테스트
- ✓ 비용·용도에 최적화된 모델 선택



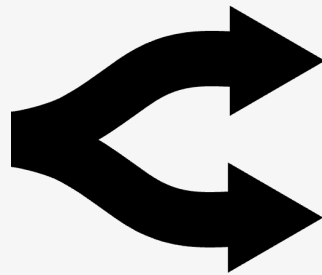


기존에 없었을까?

기존에 없었을까?



Open WebUI



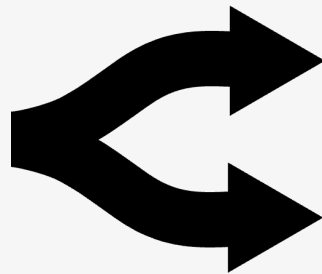
Open Router

기존에 없었을까?



Open WebUI

- ✓ 제한적인 모델 제공
- ✓ API 지원 X



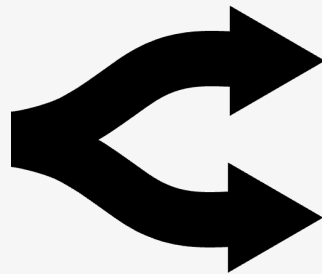
Open Router

기존에 없었을까?



Open WebUI

- ✓ 제한적인 모델 제공
- ✓ API 지원 X



Open Router

- ✓ 유료 서비스



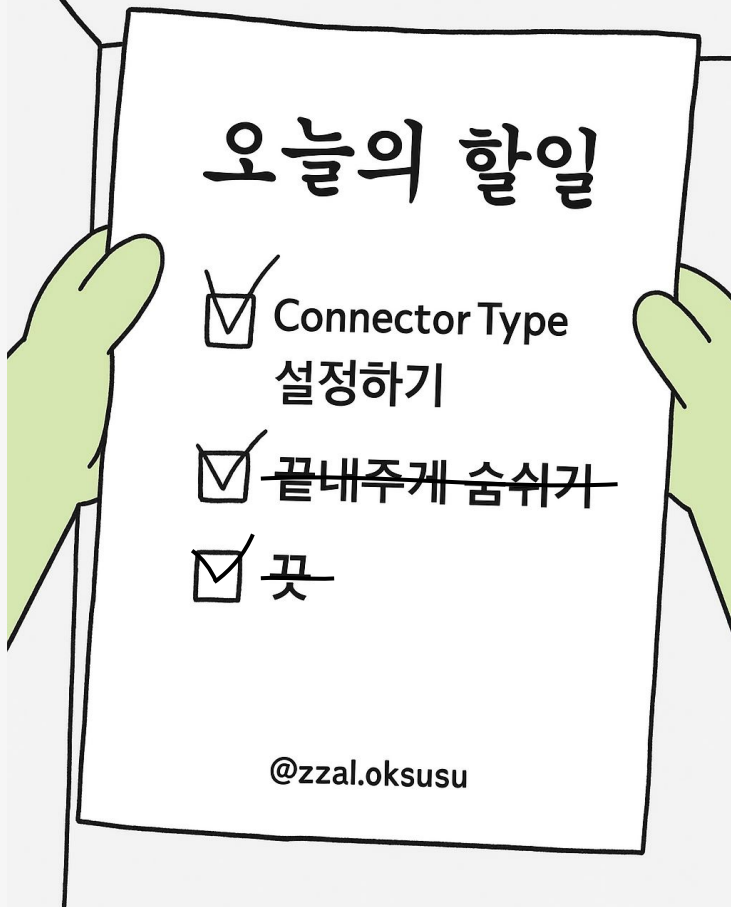
OpenChat Playground

- ✓ Local 부터 Vendor, MaaS 모델 제공자 지원
- ✓ API 지원
- ✓ OpenSource 프로젝트

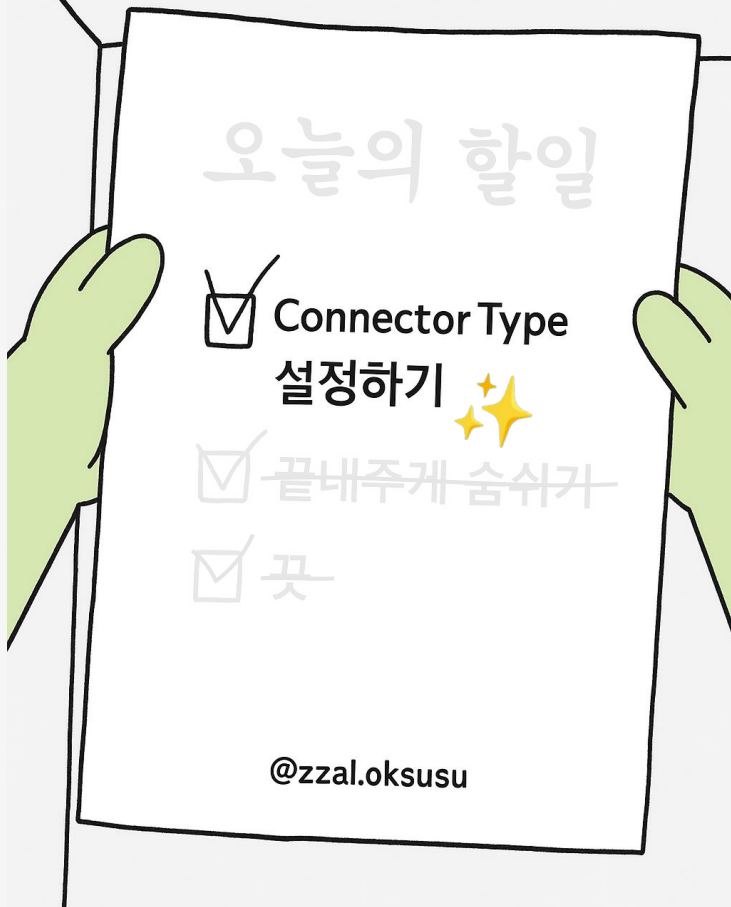


How?

OpenChat Playground,
실행하려면 ?



OpenChat Playground, 실행하려면 ?



ConnectorType 설정

 appsettings.json

 Environment Variables

 Command-Line Arguments

1 appsettings.json 파일 설정

src/OpenChat.PlaygroundApp/appsettings.json

```
{  
  ...  
  "ConnectorType": "GitHubModels",  
  ...  
}
```



```
dotnet run
```


src > OpenChat.PlaygroundApp > appsettings.json > ...

```
11
12   "ConnectorType": "GitHubModels",
13
14   "AmazonBedrock": {
15     "AccessKeyId": "{{AMAZON_BEDROCK_ACCESS_KEY_ID}}",
16     "SecretAccessKey": "{{AMAZON_BEDROCK_SECRET_ACCESS_KEY}}",
17     "Region": "{{AMAZON_BEDROCK_REGION}}",
18     "ModelId": "amazon.nova-micro-v1:0"
19   },
20
21   "AzureAIFoundry": {
22     "Endpoint": "{{AZURE_AI_FOUNDRY_ENDPOINT}}",
23     "ApiKey": "{{AZURE_AI_FOUNDRY_API_KEY}}",
24     "DeploymentName": "gpt-4o-mini"
25   },
26
27   "GitHubModels": {
28     "Endpoint": "https://models.github.ai/inference",
29     "Token": "{{GITHUB_PAT}}",
30     "Model": "openai/gpt-4o-mini"
31   },
32
33   "GoogleVertexAI": {
34     "ApiKey": "{{GOOGLE_VERTEX_AI_API_KEY}}",
35     "Model": "gemini-2.5-flash-lite"
```

```
11
12     "ConnectorType": "GitHubModels",
13
14     "AmazonBedrock": {
15         "AccessKeyId": "{AZURE_AI_FOUNDRY_API_KEY}",
16         "SecretAccessKey": "{{AMAZON_BEDROCK_SECRET_ACCESS_KEY}}",
17         "Region": "{{AMAZON_BEDROCK_REGION}}",
18         "ModelId": "amazon.nova-micro-v1:0"
19     }
20 }
```



appsettings.json

: LLM 모델 별 기본 설정



✓ 비민감 정보 설정:

- 기본 값이 미리 지정되어 있음
- }, 명령 시 인자로 전달



✓ 민감 정보 설정:

- .NET Secret Manager tool (dotnet user-secret)
- }, 명령 시 인자로 전달

```
33     "GoogleVertexAI": {
34         "ApiKey": "{{GOOGLE_VERTEX_AI_API_KEY}}",
35         "Model": "gemini-2.5-flash-lite"
```

Run on local machine

1. Make sure you are at the repository root.

```
cd $REPOSITORY_ROOT
```

2. Add GitHub Personal Access Token (PAT) for GitHub Models connection. Make sure you should replace `{{GH_PAT}}` with your GitHub PAT.

```
# bash/zsh
dotnet user-secrets --project $REPOSITORY_ROOT/src/OpenChat.PlaygroundApp \
  set GitHubModels:Token "{{GH_PAT}}"
```

```
# PowerShell
dotnet user-secrets --project $REPOSITORY_ROOT/src/OpenChat.PlaygroundApp \
  set GitHubModels:Token "{{GH_PAT}}"
```

For more details about GitHub PAT, refer to the doc, [Managing your personal access tokens](#).

3. Run the app. The default model OCP uses is [GPT-4o mini](#).



open-chat-playground



Preview README.md X



Open Chat Playground



Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



Blazor



Microsoft.
Extensions.AI

Model As A Service



Amazon
Bedrock



Azure
AI
Foundry



GitHub
Models



Google
VertexAI

Local



Docker
Model
Runner



Foundry
Local



HuggingFace



Ollama

Vendor



Anthropic
Claude



LG
AI Exaone



Naver
HyperCLOVA X



OpenAI
GPT



Upstage
Solar



PROBLEMS 8

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - OpenChat.PlaygroundApp



~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ↗2 base at 12:17:28

>

2 Environment Variables 설정



Linux / MacOS

```
export CONNECTOR_TYPE=OpenAI
```

Windows

```
set CONNECTOR_TYPE=OpenAI
```



```
dotnet run
```



open-chat-playground

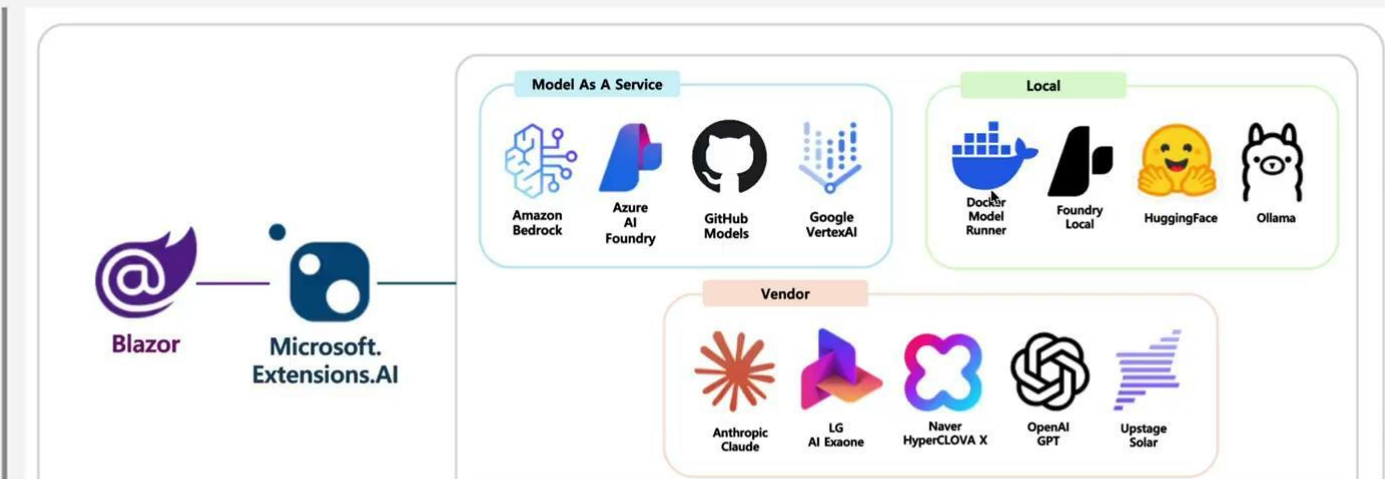


Preview README.md X



Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



PROBLEMS 8

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ↗2



```
zsh...  
zsh...
```

3 Command-Line Arguments 설정



```
dotnet run -- --connector-type OpenAI
```




open-chat-playground

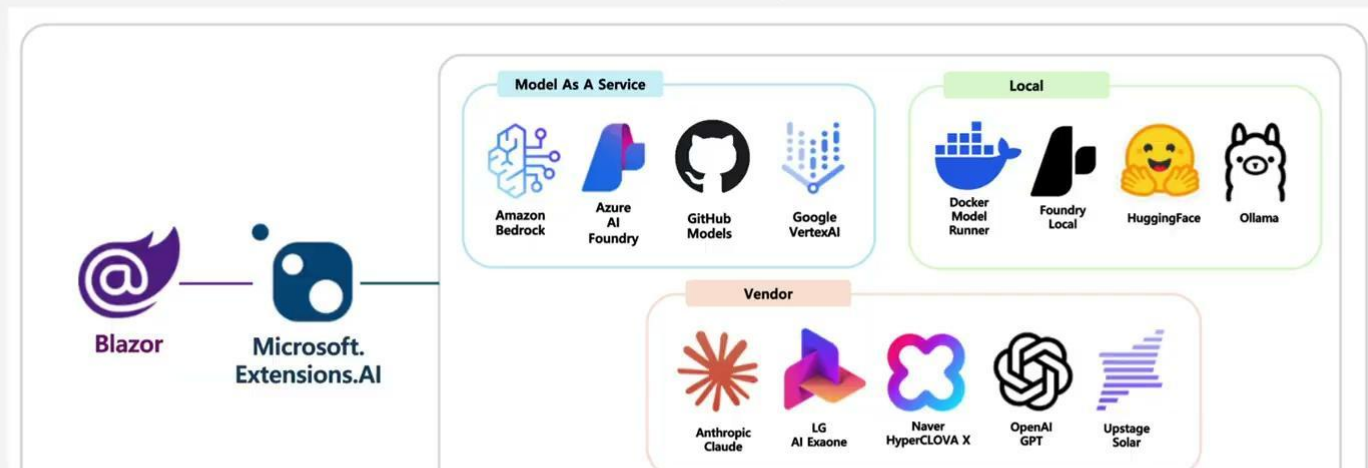


Preview README.md X



Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



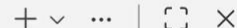
PROBLEMS 8

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



~ / g / 0 / open-chat-playground / s / OpenChat.PlaygroundApp on main ↗2



olla...

zsh...

ConnectorType 적용 순서



appsettings.json



Environment Variables



Command-Line Arguments

ConnectorType 적용 순서



appsettings.json



Environment Variables



Command-Line Arguments



ConnectorType 적용 순서

 appsettings.json



 Environment Variables

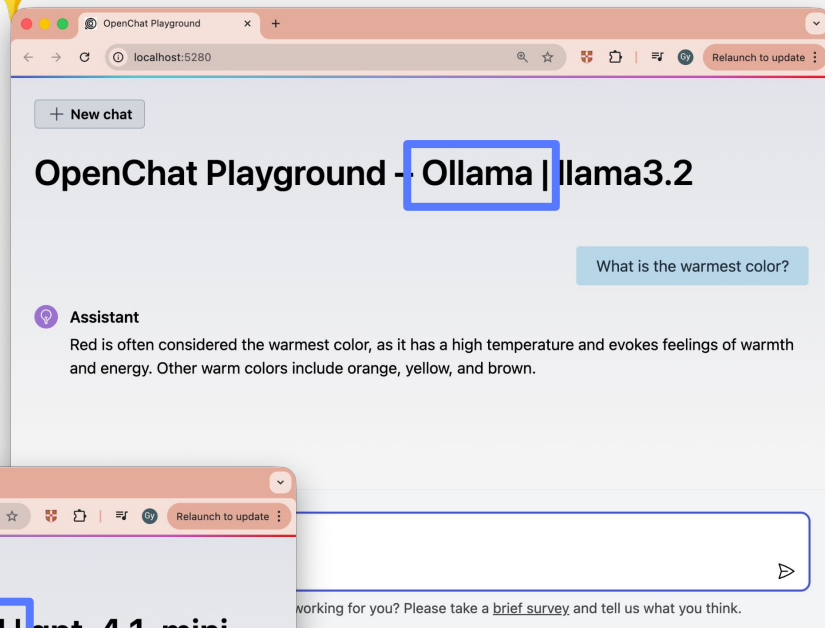
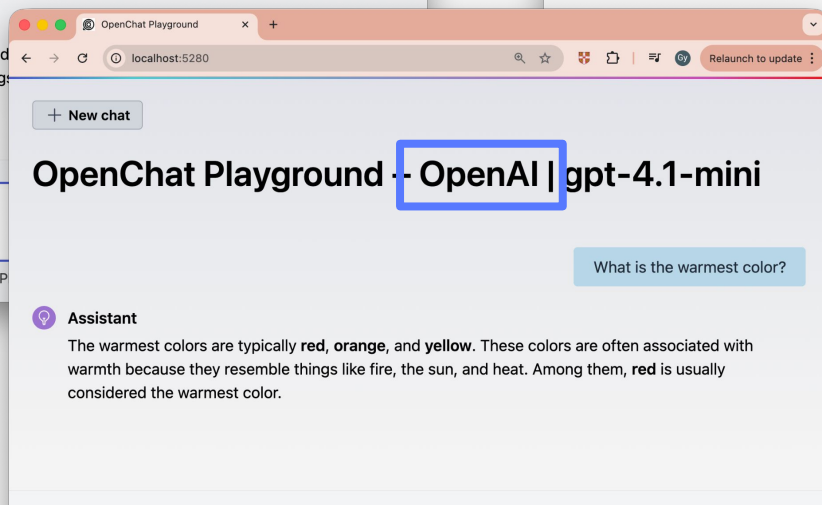
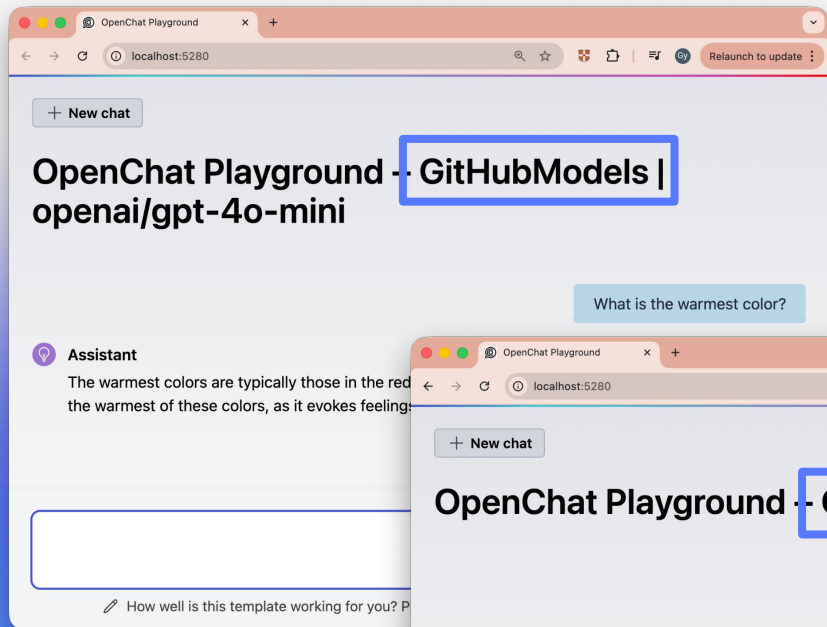


 Command-Line
Arguments





Zero-Code Switching ✨





How?

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

Amazon Bedrock SDK

Ollama SDK

Azure AI Foundry SDK

Anthropic SDK

Google Vertex AI SDK

OpenAI SDK

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조
2. 규격화의 부재
3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

호출 함수 형식, JSON 형식, API 규격 등 상이

2. 규격화의 부재

3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

2. 규격화의 부재

인증 방식, 모델 초기화 요구사항, 이미지·음성 입력 등

3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

2. 규격화의 부재

3. SDK 업데이트마다 유지보수가 발생

각 업데이트마다 모두 개별적으로 대응

🌟 IChatClient 🌟

Amazon Bedrock SDK

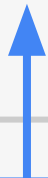
Azure AI Foundry SDK

Google Vertex AI SDK

Ollama SDK

Anthropic SDK

OpenAI SDK



🌟 IChatClient 🌟

Amazon Bedrock SDK

Ollama SDK

Microsoft.Extensions.AI
(MEAI)

Azure AI Foundry SDK

Anthropic SDK

Google Vertex AI SDK

OpenAI SDK

Microsoft.Extensions.AI

<https://learn.microsoft.com/dotnet/api/microsoft.extensions.ai>

The screenshot shows the Microsoft.Extensions.AI documentation page. The browser address bar displays the URL `learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.ai?view=net-9.0-pp`. The page header includes navigation links for **Learn**, **Documentation**, **Training**, **Q&A**, and **Topics**. Below this, there are links for **.NET**, **Languages**, **Features**, **Workloads**, **APIs**, **Troubleshooting**, and **Resources**. A **Download .NET** button is located in the top right corner. The main content area is titled **Microsoft.Extensions.AI Namespace** and contains a description: "Contains types for building and managing AI-related functionality, including chat clients, embedding generators, tools, and utilities for working with AI services." Below this, there is a section for **Classes** with an **Expand table** link. The table lists various classes and their descriptions. On the left side, there is a sidebar with a **Version** dropdown menu set to **.NET 9 (package-provided)** and a search bar. Below the search bar, a list of classes is shown, including **Microsoft.Extensions.AI**, **AdditionalPropertiesDictionary**, **AdditionalPropertiesDictionary<TValue>.Enumerator**, **AdditionalPropertiesDictionary<TValue>**, **AIAnnotation**, **AIContent**, **AIFunction**, **AIFunctionArguments**, **AIFunctionDeclaration**, **AIFunctionFactory**, **AIFunctionFactoryOptions**, **AIFunctionFactoryOptions.ParameterBindingOptions**, **AIJsonSchemaCreateContext**, **AIJsonSchemaCreateOptions**, **AIJsonSchemaTransformCache**, **AIJsonSchemaTransformContext**, **AIJsonSchemaTransformOptions**, **AIJsonUtilities**, **AITool**, and **AnnotatedRegion**. On the right side, there is a section titled **In this article** with links to **Classes**, **Structs**, **Interfaces**, and **Enums**. Below this, there is a **Was this page helpful?** section with **Yes** and **No** buttons.

Version: .NET 9 (package-provided)

Search: Search

Microsoft.Extensions.AI

- > AdditionalPropertiesDictionary
- > AdditionalPropertiesDictionary<TValue>.Enumerator
- > AdditionalPropertiesDictionary<TValue>
- > AIAnnotation
- > AIContent
- > AIFunction
- > AIFunctionArguments
- > AIFunctionDeclaration
- > AIFunctionFactory
- > AIFunctionFactoryOptions
- > AIFunctionFactoryOptions.ParameterBindingOptions
- > AIJsonSchemaCreateContext
- > AIJsonSchemaCreateOptions
- > AIJsonSchemaTransformCache
- > AIJsonSchemaTransformContext
- > AIJsonSchemaTransformOptions
- > AIJsonUtilities
- > AITool
- > AnnotatedRegion

Learn / .NET / API browser / Ask Learn Focus mode

Microsoft.Extensions.AI Namespace

Contains types for building and managing AI-related functionality, including chat clients, embedding generators, tools, and utilities for working with AI services.

Classes

Expand table

AdditionalPropertiesDictionary	Provides a dictionary used as the AdditionalProperties dictionary on Microsoft.Extensions.AI objects.
AdditionalPropertiesDictionary<TValue>	Provides a dictionary used as the AdditionalProperties dictionary on Microsoft.Extensions.AI objects.
AIAnnotation	Represents an annotation on content.
AIContent	Represents content used by AI services.
AIFunction	Represents a function that can be described to an AI service and invoked.
AIFunctionArguments	Represents arguments to be used with InvokeAsync(AIFunctionArguments, CancellationToken) .
AIFunctionDeclaration	Represents a function that can be described to an AI service.
AIFunctionFactory	Provides factory methods for creating commonly-used implementations of AIFunction .
AIFunctionFactoryOptions	Represents options that can be provided when creating an AIFunction from a

In this article

- Classes
- Structs
- Interfaces
- Enums

Was this page helpful?

Yes No

Microsoft.Extensions.AI – IChatClient

<https://learn.microsoft.com/dotnet/api/microsoft.extensions.ai>

The screenshot shows the Microsoft Learn documentation page for the `IChatClient` interface. The browser address bar shows the URL `learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.ai.ichatclient?view=net-9.0-pp`. The page has a navigation bar with links for `.NET`, `Learn`, `Documentation`, `Training`, `Q&A`, and `Topics`. A search bar and a `Sign in` link are also present. The main content area is titled `IChatClient Interface` and includes a `Definition` section. The definition states that the interface is in the `Microsoft.Extensions.AI` namespace, assembly `Microsoft.Extensions.AI.Abstractions.dll`, package `Microsoft.Extensions.AI.Abstractions v10.0.0`, and source `IChatClient.cs`. It represents a chat client. The code snippet shows the interface definition in C#: `public interface IChatClient : IDisposable`. The `Derived` section lists `Microsoft.Extensions.AI.DelegatingChatClient`, and the `Implements` section lists `IDisposable`. The `Remarks` section notes that applications must consider risks such as prompt injection attacks, data sizes, and the number of messages sent to the underlying provider or returned from it. A sidebar on the left shows a tree view of the `IChatClient` interface and its methods. A right sidebar contains links for `Definition`, `Remarks`, `Methods`, `Extension Methods`, `Applies to`, and `See also`. A `Was this page helpful?` section with `Yes` and `No` buttons is at the bottom right.

Version: .NET 9 (package-provided)

Learn / .NET / API browser /

IChatClient Interface

Definition

Namespace: Microsoft.Extensions.AI
Assembly: Microsoft.Extensions.AI.Abstractions.dll
Package: Microsoft.Extensions.AI.Abstractions v10.0.0
Source: IChatClient.cs

Represents a chat client.

```
C#  
public interface IChatClient : IDisposable
```

Derived: Microsoft.Extensions.AI.DelegatingChatClient

Implements: IDisposable

Remarks

Applications must consider risks such as prompt injection attacks, data sizes, and the number of messages sent to the underlying provider or returned from it. Unless a specific `IChatClient` implementation explicitly documents safeguards for these concerns, the application is expected to implement appropriate protections.

In this article

- Definition
- Remarks
- Methods
- Extension Methods
- Applies to
- See also

Was this page helpful?

Yes No



IChatClient.cs

```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```

IChatClient.cs

```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```


IChatClient.cs

```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

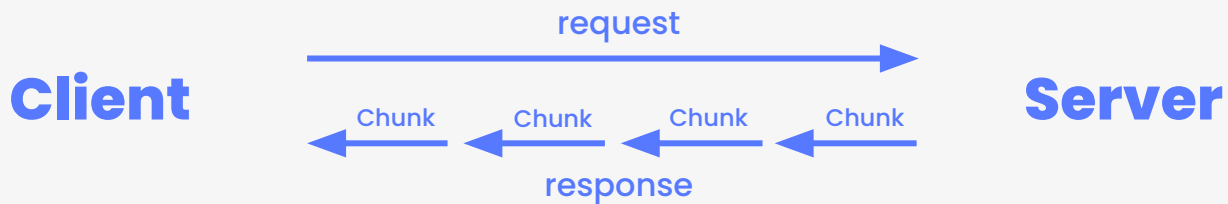
    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```

```
Task<ChatResponse> GetStreamingResponse(...)
```



```
IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(...)
```



LLM Output: Streaming vs Non-Streaming

 Non-Streaming Output

 Streaming Output

EX. IChatClient::GetResponseAsync



```
using Microsoft.Extensions.AI;  
using OllamaSharp;  
  
IChatClient chatClient = new OllamaApiClient(  
    new Uri("http://localhost:11434/"), "phi3:mini");  
  
var result = await chatClient.GetResponseAsync("What is AI?");  
Console.WriteLine(result);
```

EX. IChatClient::GetResponseAsync



```
using Microsoft.Extensions.AI;  
using OllamaSharp;  
  
IChatClient chatClient = new OllamaApiClient(  
    new Uri("http://localhost:11434/"), "phi3:mini");  
  
var result = await chatClient.GetResponseAsync("What is AI?");  
Console.WriteLine(result);
```

EX. IChatClient::GetStreamingResponseAsync



```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
    new Uri("http://localhost:11434/"), "phi3:mini");

await foreach (ChatResponseUpdate item in chatClient.GetStreamingResponseAsync("What is AI?"))
{
    Console.Write(item.Text);
}
```

EX. IChatClient::GetStreamingResponseAsync



```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
    new Uri("http://localhost:11434/"), "phi3:mini");

await foreach (ChatResponseUpdate item in chatClient.GetStreamingResponseAsync("What is AI?"))
{
    Console.Write(item.Text);
}
```



OpenChat Playground에서는
MEAI를 어떻게 활용했을까 ?

OCP에서는 어떻게 활용했을까 ? - Key Idea 💡

Key Idea Pseudo Code

```
using Microsoft.Extensions.AI;  
  
IChatClient chatClient = ConnectorFactory.Create(ConnectorType);
```

OCP에서는 어떻게 활용했을까 ? - Overview

src/OpenChat.PlaygroundApp/Program.cs

```
var builder = WebApplication.CreateBuilder(args);  
...  
  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
  
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);  
...  
  
builder.Services.AddChatClient(chatClient)  
                .UseFunctionInvocation()  
                ...;  
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

OCP에서는 어떻게 활용했을까 ? - Overview



```
var builder = WebApplication.CreateBuilder(args);
```



ASP.NET Core 애플리케이션 빌더 생성

```
var settings = ArgumentOptions.Parse(builder.Configuration, args);
```

```
...
```

```
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
```

```
...
```

```
builder.Services.AddChatClient(chatClient)
    .UseFunctionInvocation()
    ...;
```

```
...
```

```
builder.Services.AddScoped<IChatService, ChatService>();
```

```
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);  
...
```

```
var settings = ArgumentOptions.Parse(builder.Configuration, args);
```

↑ 설정 파싱하여 AppSettings 객체 생성

```
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);  
...
```

```
builder.Services.AddChatClient(chatClient)  
                .UseFunctionInvocation()  
                ...;
```

```
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);  
...  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
```

↑ 파싱된 설정에 따라 적절한 AI 모델 커넥터를 선택하고,

```
builder.Services.AddSingleton<IChatClient>(() => chatClient);  
builder.Services.AddScoped<IChatService, ChatService>();  
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

OCP에서는 어떻게 활용했을까 ? - Overview

```
var builder = WebApplication.CreateBuilder(args);  
...  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
var 생성된 chatClient를 DI 컨테이너에 connector.CreateChatClientAsync(settings);  
↓ 채팅 클라이언트 서비스 (싱글톤) 등록  
builder.Services.AddChatClient(chatClient)  
    .UseFunctionInvocation()  
    ...;  
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

OCP에서는 어떻게 활용했을까 ? - Overview

```
var builder = WebApplication.CreateBuilder(args);  
...  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);  
...
```

```
builder.Services.AddChatClient(chatClient)
```

`IChatService` 인터페이스와 `ChatService` 구현체를



스코프드 서비스로 등록하여 채팅 로직 캡슐화

```
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

OCP에서는 어떻게 활용했을까? - Overview



```
var builder = WebApplication.CreateBuilder(args);  
...  
  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
  
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);  
...  
  
builder.Services.AddChatClient(chatClient)  
                .UseFunctionInvocation()  
                ...;  
  
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```


OCP에서는 어떻게 활용했을까 ? - Overview

```
var builder = WebApplication.CreateBuilder(args);  
...  
var settings = ArgumentOptions.Parse(builder.Configuration, args);  
...  
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);  
...  
builder.Services.AddChatClient(chatClient)  
    .UseFunctionInvocation()  
    ...;  
...  
builder.Services.AddScoped<IChatService, ChatService>();  
...
```

Factory Template Pattern: *LanguageModelConnector*

<<Abstract>>

LanguageModelConnector

+ **CreateChatClientAsync(): IChatClient**

- <<Abstract>> **GetChatClientAsync(): IChatClient**

Factory Template Pattern: *LanguageModelConnector*

«Abstract»

LanguageModelConnector

+**Create()**: IChatClient

+ «Abstract» **GetClient()**: IChatClient

Factory Template Pattern: *LanguageModelConnector*

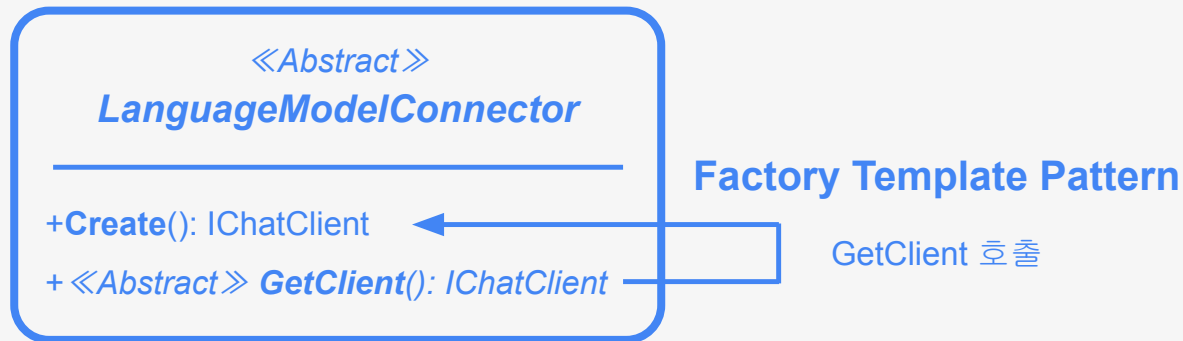
«Abstract»

LanguageModelConnector

+**Create()**: IChatClient

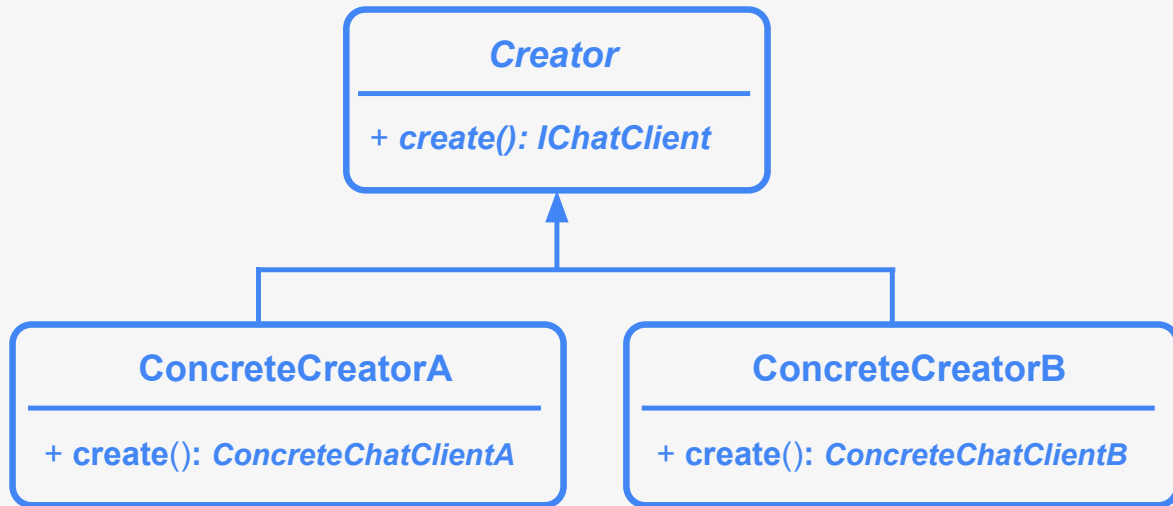
+ «Abstract» **GetClient()**: IChatClient

Factory Template Pattern: *LanguageModelConnector*



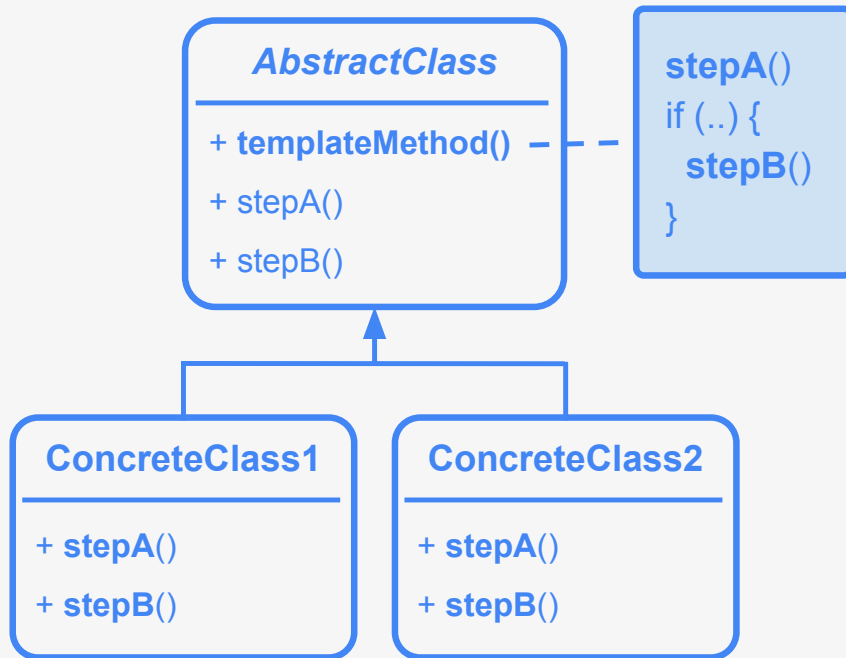
Factory Template Pattern: *LanguageModelConnector*

Factory Method Pattern

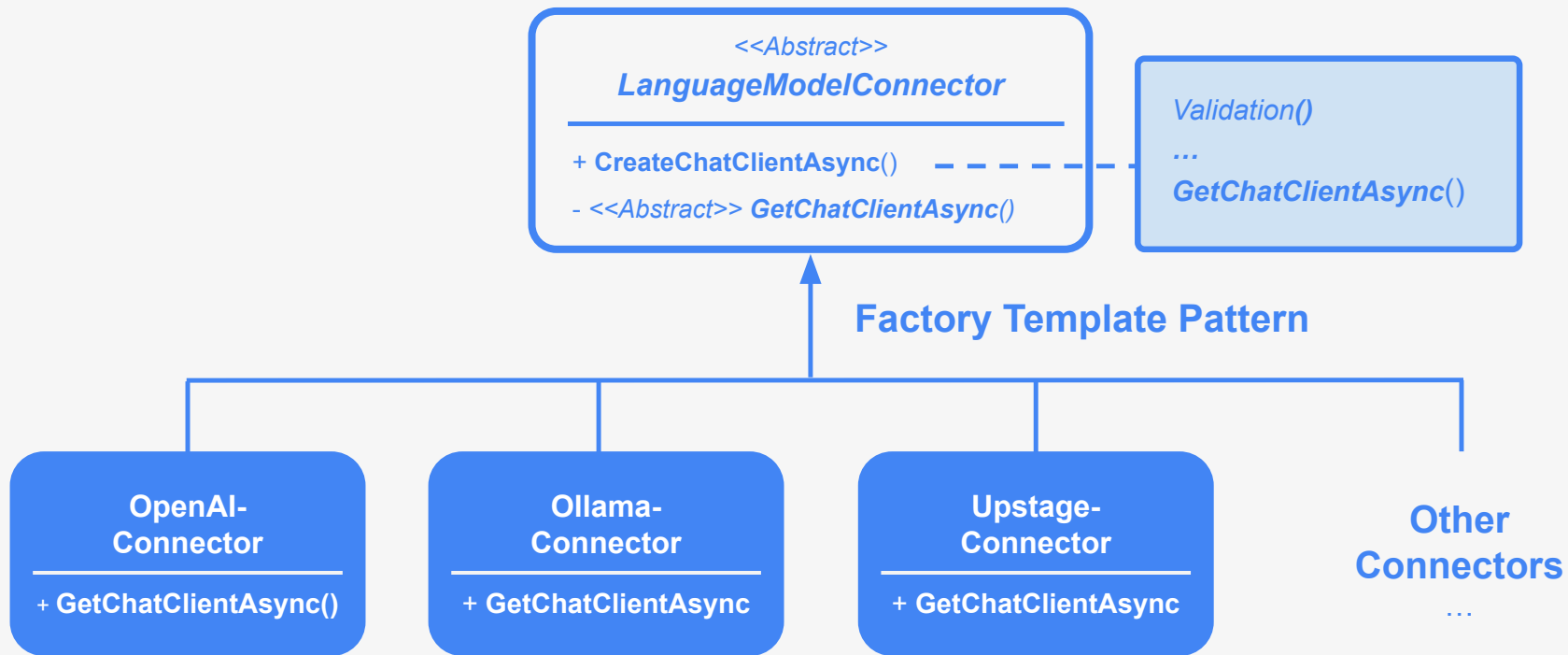


Factory Template Pattern: *LanguageModelConnector*

Template Method Pattern



Factory Template Pattern: *LanguageModelConnector*



Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

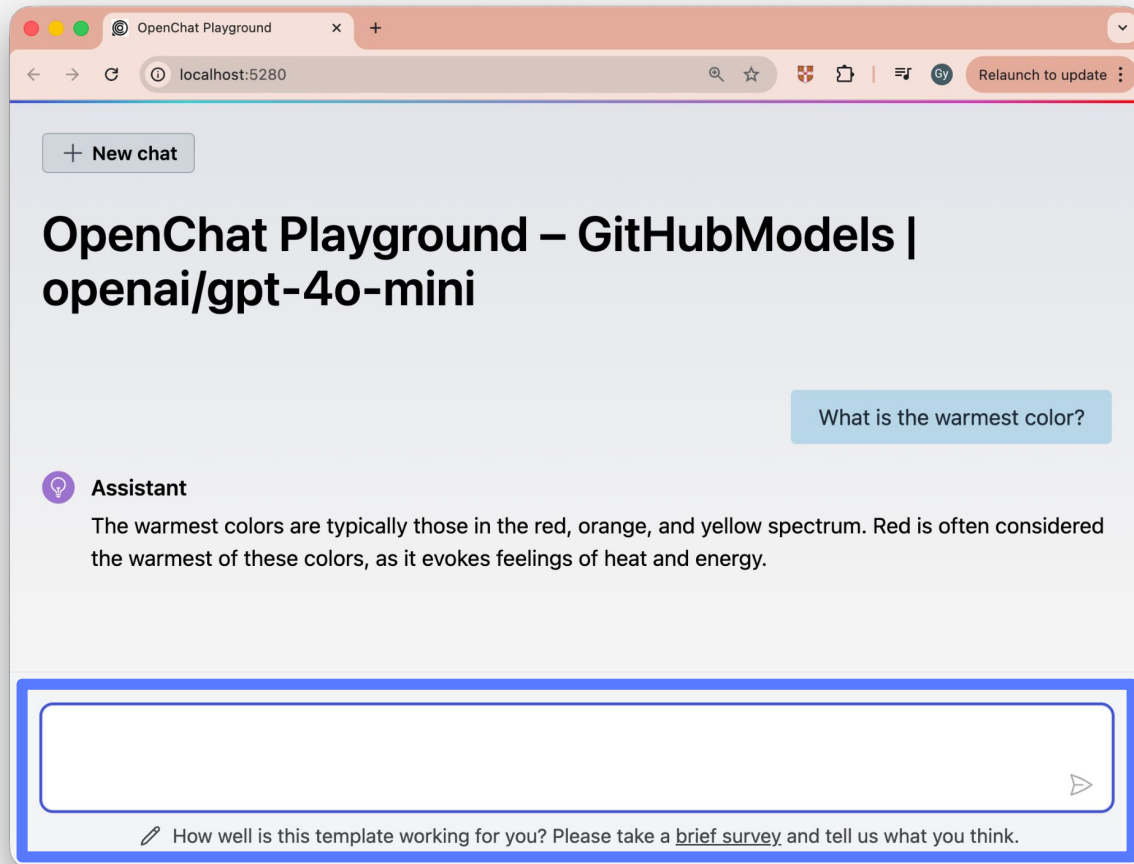
        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

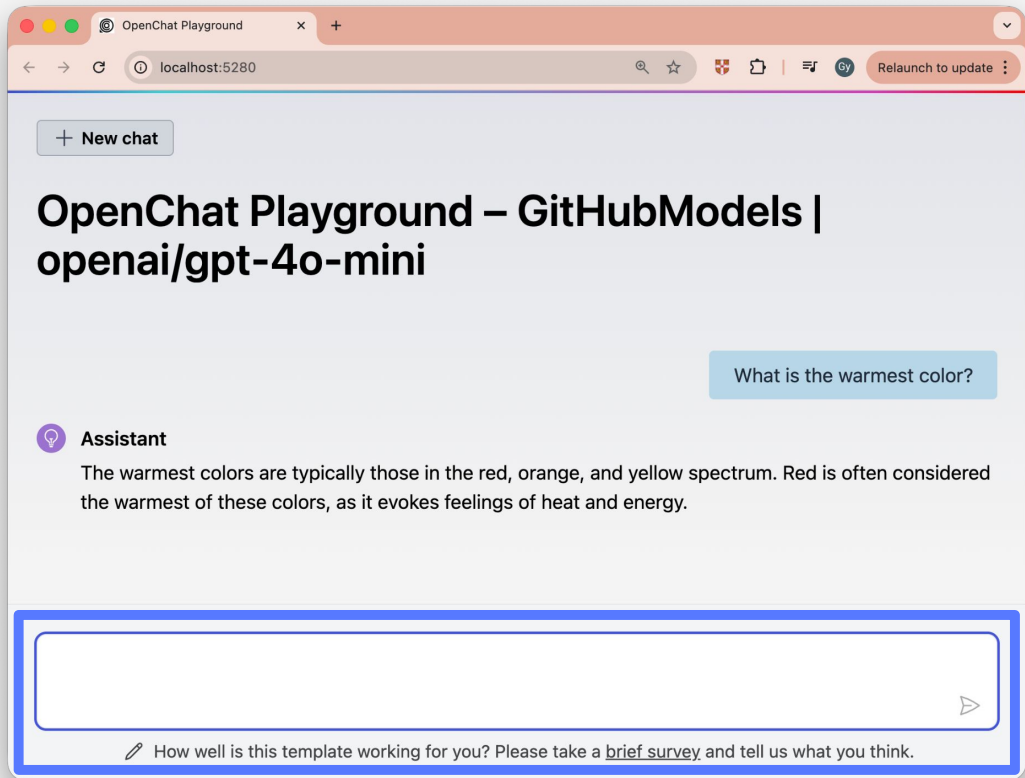
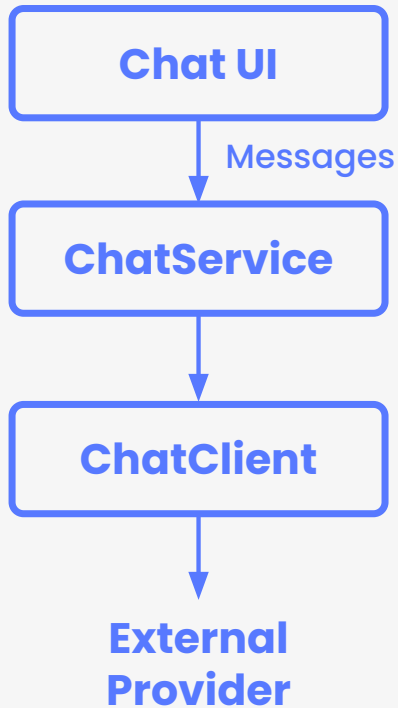


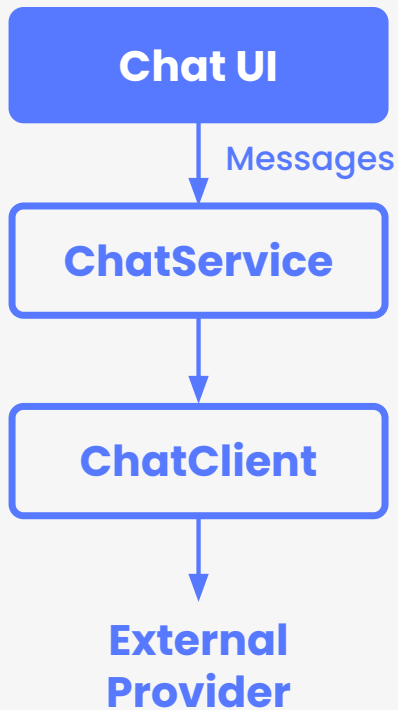
잘 만들어진 LLM Provider,
사용자의 요청에 어떻게
동작할까 ?

Flow: Web UI → External LLM Provider









Chat.razor

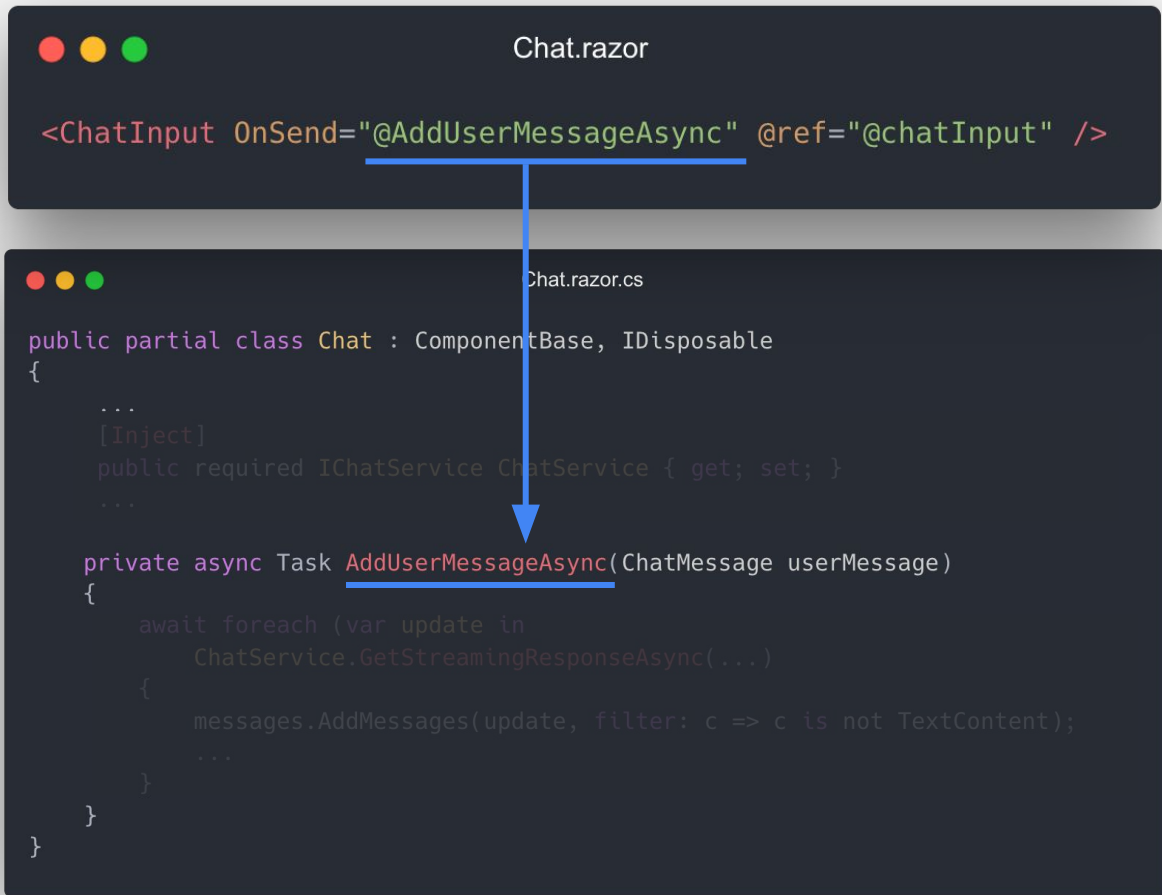
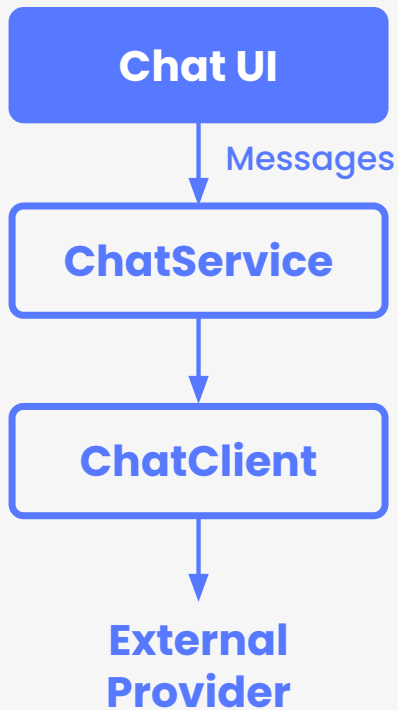
```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

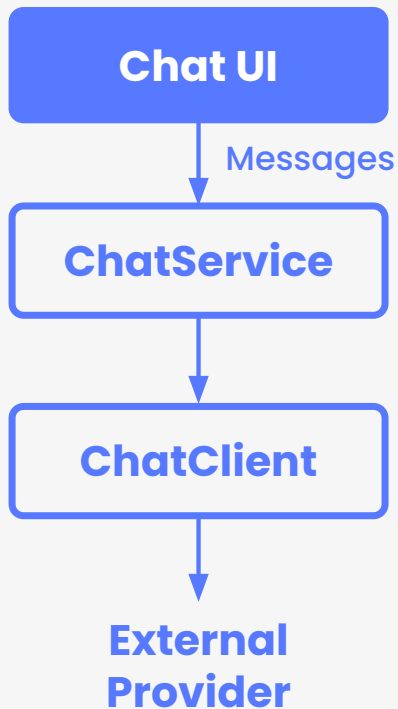


Chat.razor.cs

```
public partial class Chat : ComponentBase, IDisposable
{
    ...
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...))
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```





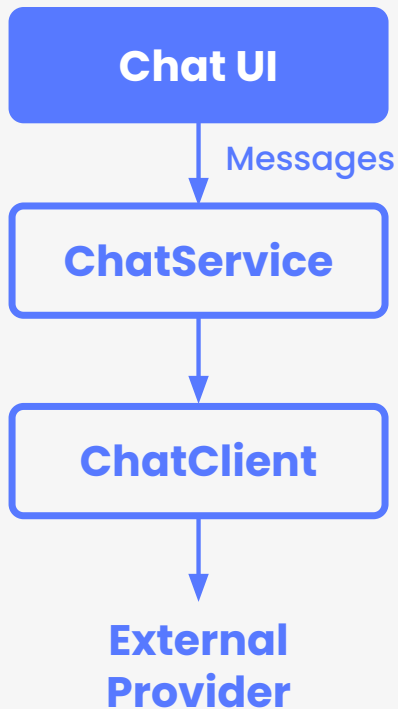
```
Chat.razor

<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

```
Chat.razor.cs

public partial class Chat : ComponentBase, IDisposable
{
    UI에서 ChatService를 의존성 주입 받아 사용
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...))
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```



Chat.razor

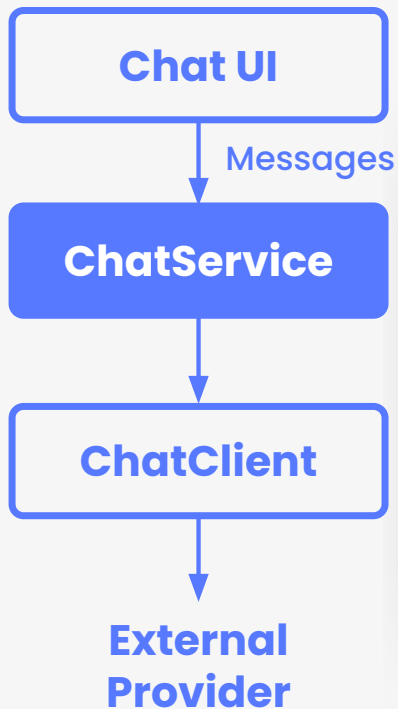
```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

Chat.razor.cs

```
public partial class Chat : ComponentBase, IDisposable
{
    ...
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...))
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```

ChatService가 실제 유저 메시지 처리 전담

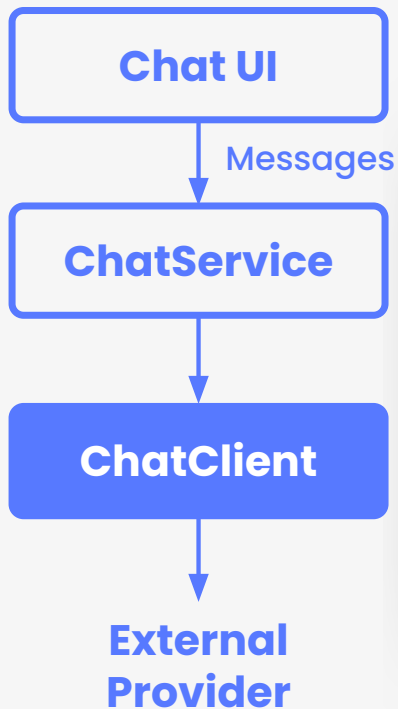


DI 컨테이너가 **ChatService**를 생성할 때
미리 생성된 **ChatClient** 인스턴스를 생성자 인자로 주입

```
ChatService.cs

public class ChatService(IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    )
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```

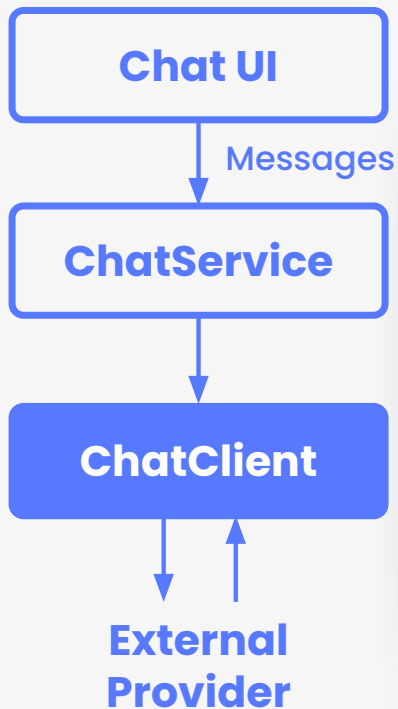


각 LLM Provider 별 Connector 에서 생성한
IChatClient 구현체

ChatService.cs

```
public class ChatService(IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```



ChatService.cs

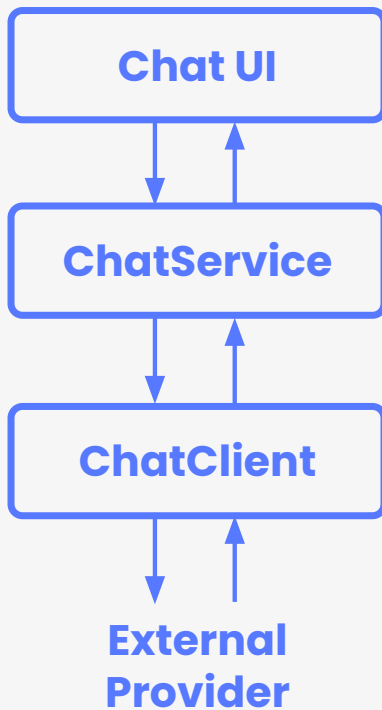
```
public class ChatService(IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```

LLM Provider Connector가 생성한
ChatClient 구현체를 사용해
스트리밍 Chat Completion 호출



사용자의 요청부터 LLM Provider 까지





백엔드 API가 필요하나요 ?





백엔드 **API**가 필요하да구요 ?

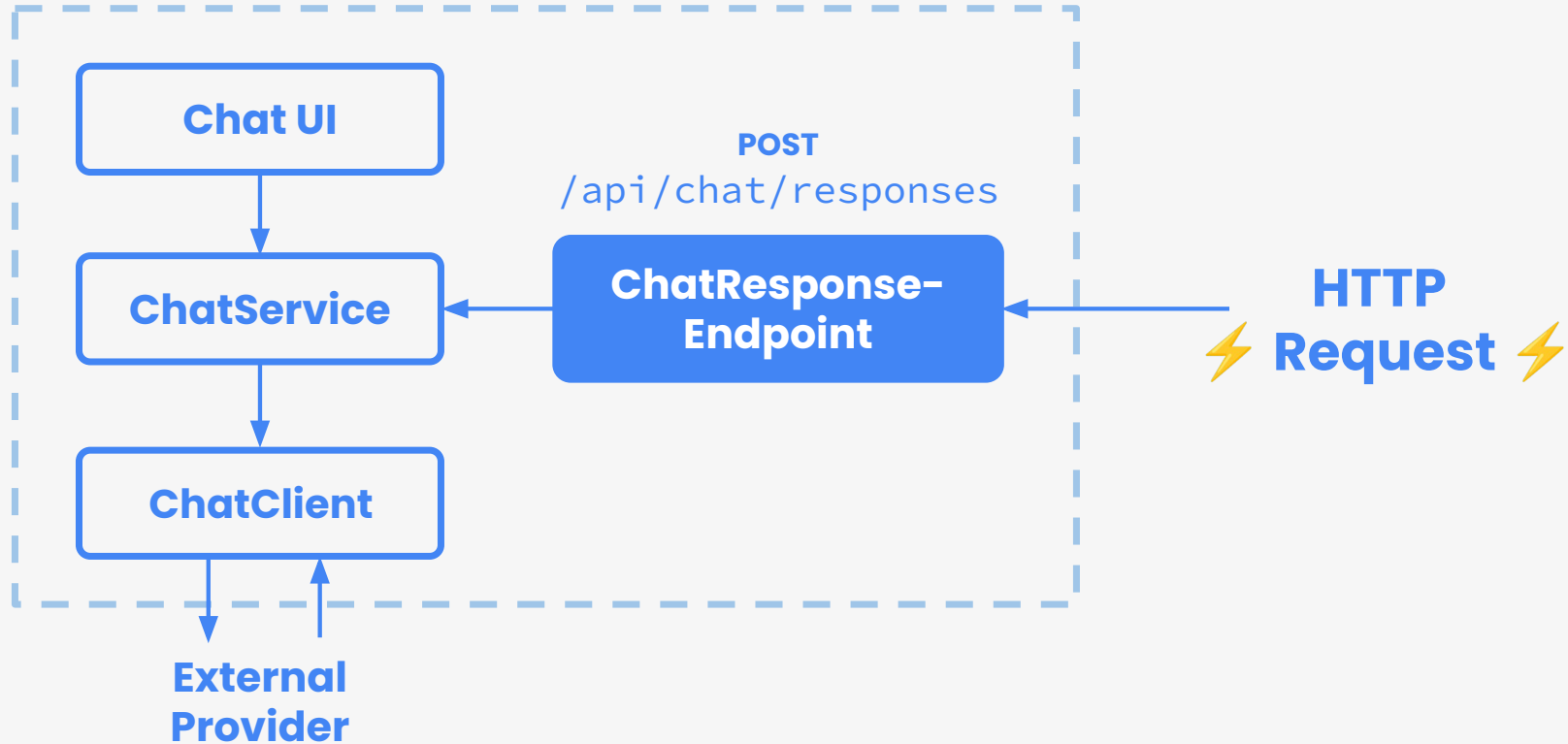
여기 있습니다 🤝



/api/chat/responses



PlaygroundApp



Demo

Chat.razor.cs

```
curl -N -X POST http://localhost:5280/api/chat/responses \
  -H "Content-Type: application/json" \
  -d '[
    {
      "role": "system",
      "message": "You are a helpful assistant."
    },
    {
      "role": "user",
      "message": "What is the warmest color?"
    }
  ]'
```

CodeFileEditSelectionViewGoRunTerminalWindowHelp

open-chat-playground

Preview README.md ×curl-example.txt UREADME.md

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.

PROBLEMS 8

OUTPUT

DEBUG CONSOLE

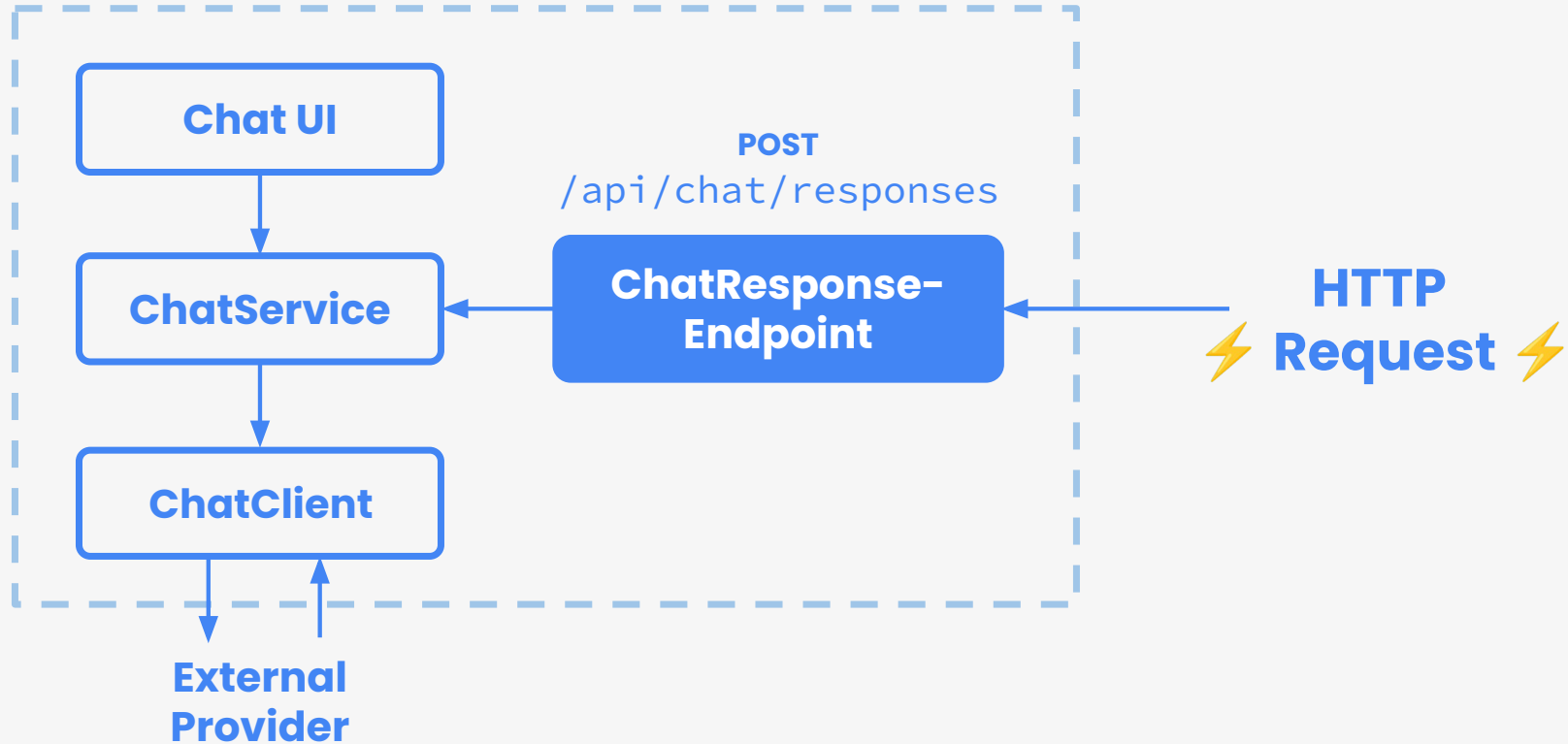
TERMINAL

PORTS

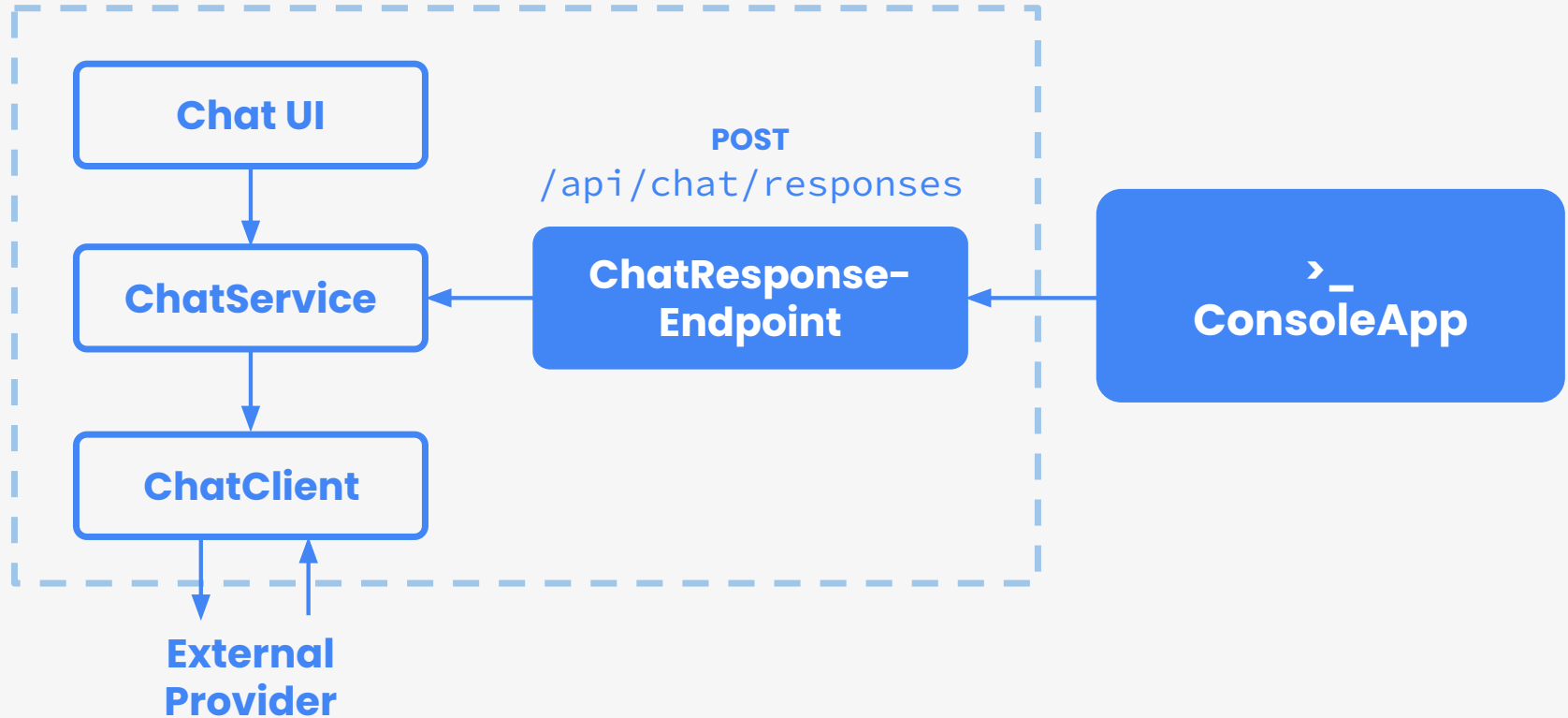
main* 0 ↓ 2 ↑ 0 ⚠ 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git Graph

olla...
zsh...

PlaygroundApp



PlaygroundApp





Project Structure

open-chat-playground/

├── src/

│ ├── **OpenChat.PlaygroundApp/**

│ │ ├── Components/

│ │ ├── Abstractions/

│ │ ├── Connectors/

│ │ ├── Configurations/

│ │ ├── Endpoints/

│ │ └── Services/

│ └── **OpenChat.ConsoleApp/**

│ ├── Clients/

│ ├── Services/

│ └── Options/

├── test/

├── docs/

├── infra/

└── README.md

웹 애플리케이션

Blazor 컴포넌트

추상화 레이어

LLM 제공업체 커넥터

설정 클래스

API 엔드포인트

비즈니스 로직

CLI 애플리케이션

HTTP 클라이언트

서비스 로직

설정 옵션

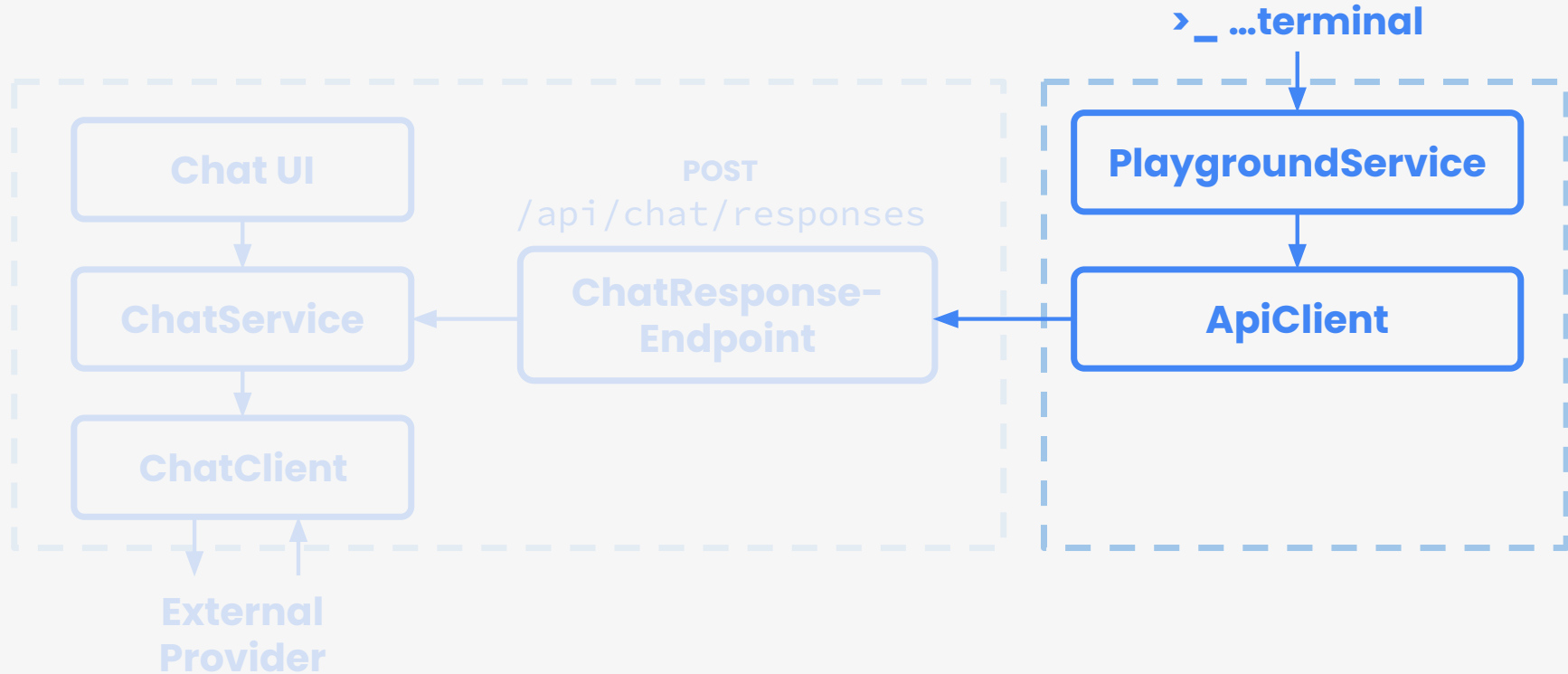
테스트

문서

Azure 인프라

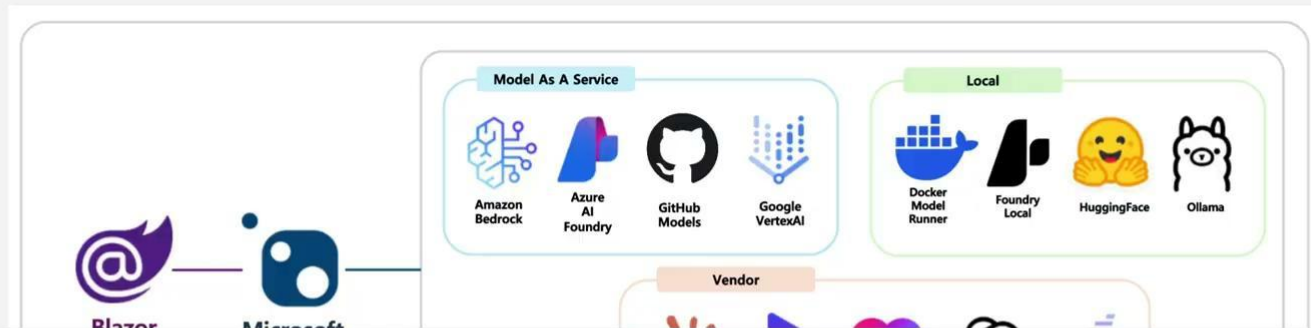
PlaygroundApp

ConsoleApp



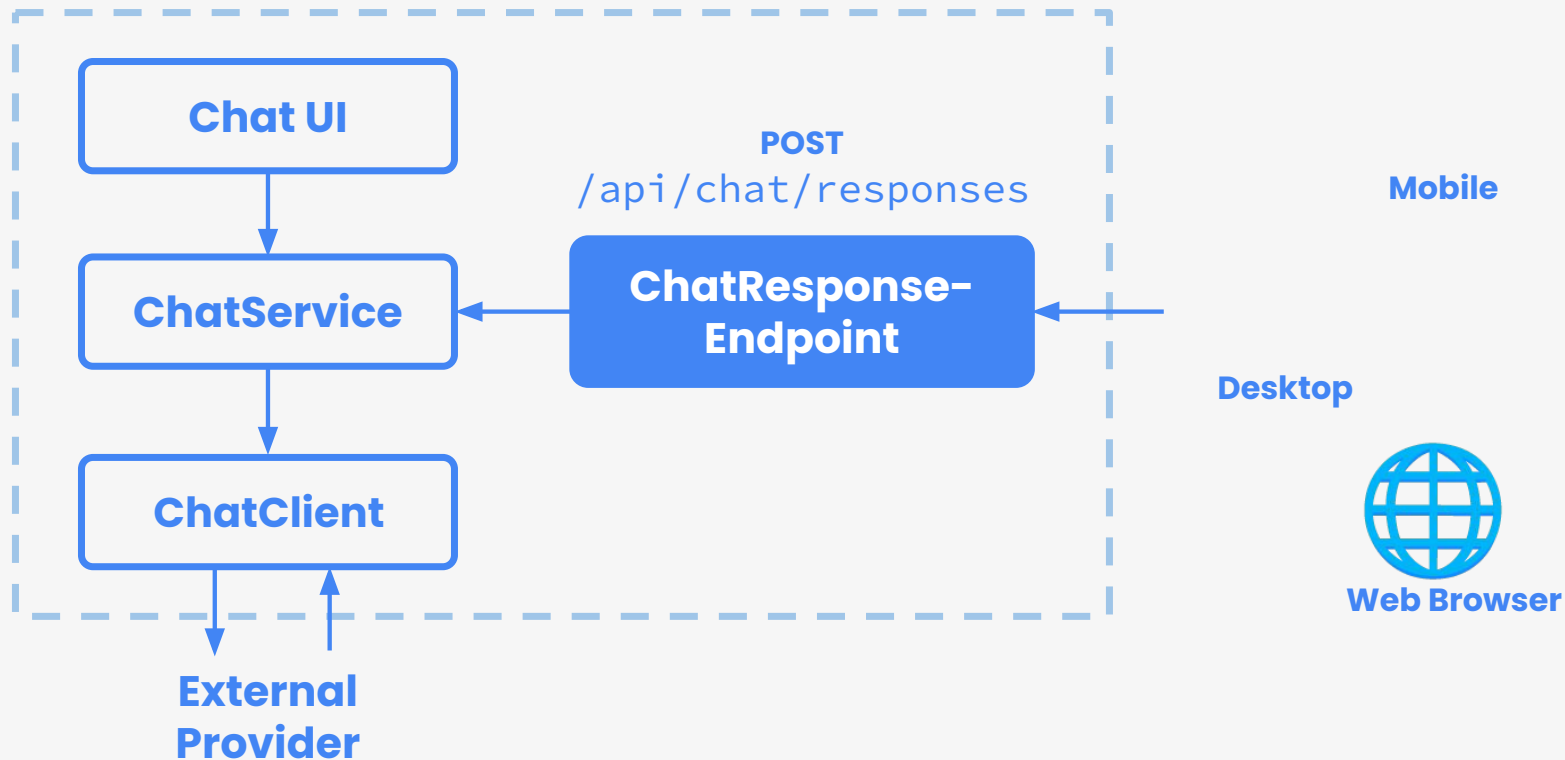
Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.





**One API.
Any Device.** ⚡





빌드 환경



Local 환경



Container 환경



Azure Container App 배포



Project Structure

📁 open-chat-playground/

├─ src/

│ └─ OpenChat.PlaygroundApp/

│ └─ OpenChat.ConsoleApp/

├─ test/

├─ docs/

├─ **infra/**

└─ README.md

웹 애플리케이션

CLI 애플리케이션

테스트

문서

Azure 인프라



Project Structure

```
open-chat-playground/  
├── src/  
│   ├── OpenChat.PlaygroundApp/  
│   └── OpenChat.ConsoleApp/  
├── test/  
├── docs/  
├── infra/  
└── README.md
```

```
▼ infra  
  ▼ modules  
    🔧 fetch-container-image.bicep  
    {} abbreviations.json  
    🔧 main.bicep  
    {} main.parameters.json  
    🔧 resources.bicep
```



빌드 환경



Local 환경



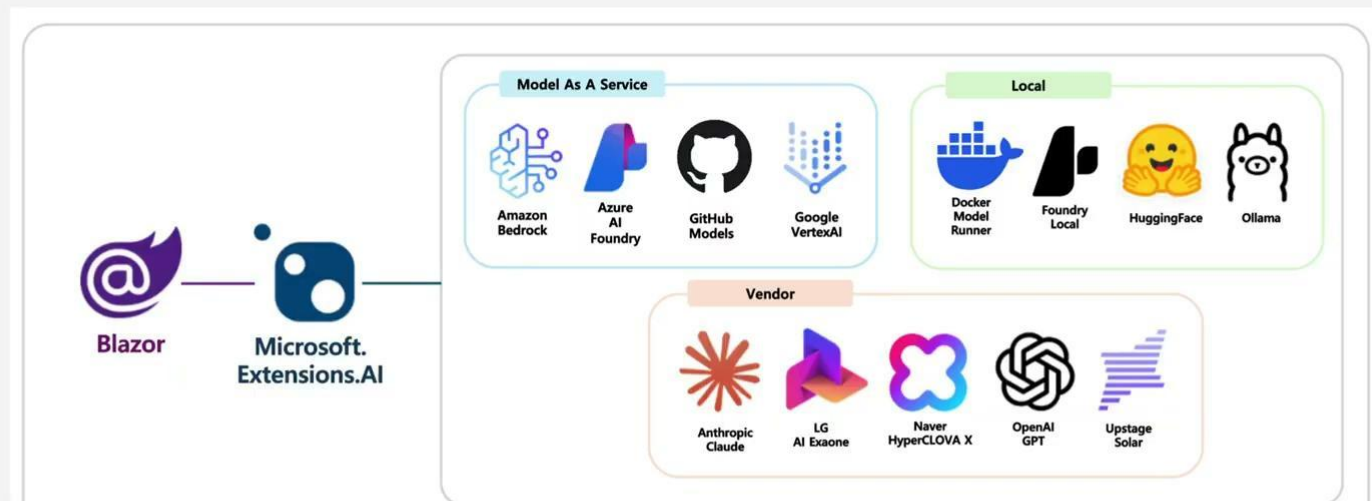
Container 환경



Azure Container App 배포

 Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.





빌드 환경



Local 환경



Container 환경



Azure Container App 배포



Roadmap



Roadmap



LLM 제공 지원 확대



Real-time LLM 교체



Aspire Orchestration



MCP 서버 지원



Roadmap



LLM 제공 지원 확대



Real-time LLM 교체



Aspire Orchestration



MCP 서버 지원



Roadmap

- ✓ LLM 제공 지원 확대
- ✓ Real-time LLM 교체
- ✓ Aspire Orchestration
- ✓ MCP 서버 지원



Roadmap

- ✓ LLM 제공 지원 확대
- ✓ Real-time LLM 교체
- ✓ Aspire Orchestration
- ✓ MCP 서버 지원



Open Source



Open Source



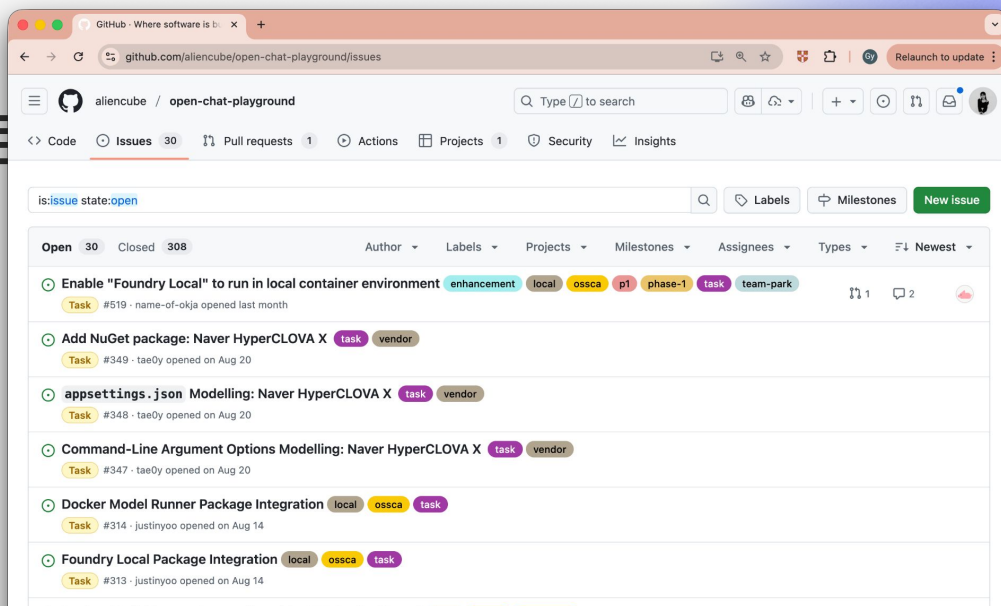
누구나 기여 가능



현행화된 문서화



지속적인 업데이트





OpenChat Playground

<https://bit.ly/openchat-playground>





더 알아보고 싶다면

<https://learn.microsoft.com/dotnet/ai>

<https://learn.microsoft.com/dotnet/ai/microsoft-extensions-ai>

EOD